

**UNIVERSIDADE ESTÁCIO DE SÁ
CAMPUS NOVA AMÉRICA**

Projeto de cadastramento para clientes em Java

**Daniel Vinicius Lima Caravelos
Daniel Lima Pacheco do Nascimento
Erick Freire dos Santos
Isaac Blanco Pacheco
Sidney Rodrigues da Costa Júnior**

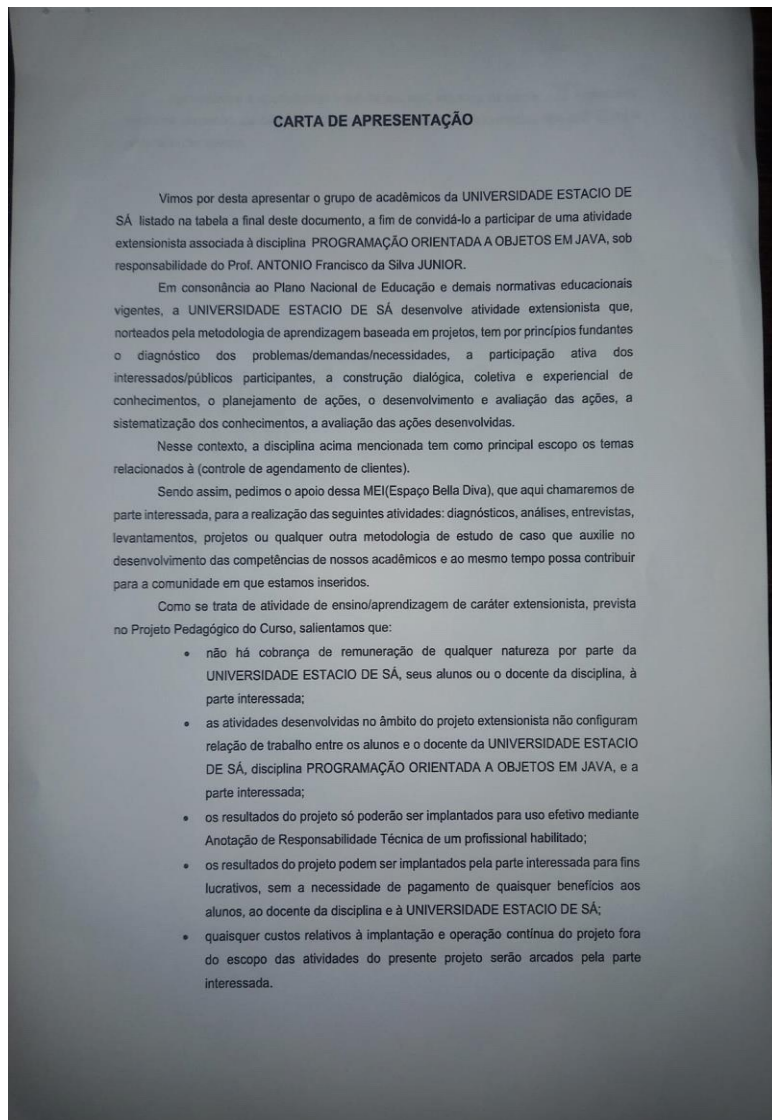
Professor: Antônio Francisco da Silva Junior

**2024
Rio de Janeiro/RJ**

- **DIAGNÓSTICO E TEORIZAÇÃO**

- Identificação das partes interessadas e parceiros

A parte interessada é a esteticista chamada Cristiane Gabriel Freire dos Santos (Classe média baixa, nível superior incompleto, feminino, 48 anos) e dona do espaço Bella Diva.



Aproveitamos a oportunidade e solicitamos que, em caso de aceite, seja formalizado, mediante assinatura da Carta de Autorização, as atividades e informações que o(s) aluno(s) poderá(ão) ter acesso.

Cristiane P. S. dos Santos

Nome do responsável
CNPJ: 40535435000155
Data: 20/09/2024

Desde já nos colocamos à sua disposição para quaisquer esclarecimentos.

Grupo de Alunos
Erick Freire dos Santos Matrícula: 202302249647
Daniel Vinicius Lima Caravelos Matrícula: 202302249612
Daniel Lima Pacheco do Nascimento Matrícula: 202209189117
Isaac Blanco Pacheco Matrícula: 202303405804
Sidney Rodrigues da Costa Junior Matrícula: 202203423584

Atenciosamente,


ANTONIO Francisco da Silva JUNIOR
Docente disc. PROG ORIENTAÇÃO EM JAVA
Semestres 2024.2 - Matrícula: 1015863

Rio de Janeiro 20, Setembro de 2024



- **Problemática e/ou problemas identificados**

O espaço Bella Diva é um estabelecimento de atendimento para serviços de estética, e a mesma acaba possuindo muitas dificuldades para organizar o agendamento de cada atendimento feito pelos mesmos e também na organização no cadastro das pessoas X serviços escolhidos.

- **Justificativa**

Com a problemática identificada, o grupo decidiu utilizar as aplicações de Java para criar um sistema de cadastro para auxiliar o estabelecimento, com isso facilitando a organização do lugar de forma simples e direta.

Sendo um sistema onde no sistema o cadastro é feito utilizando o nome, cpf e o procedimento escolhido pelo cliente, com isso o sistema irá salvar todos os dados no banco de dados SQL. (Lembrando que no sistema é possível ver todos os clientes e pedidos cadastrados, com isso a pessoa responsável consegue adicionar, editar e remover todos os dados cadastrados ou que a mesma decida cadastrar.

- **Objetivos/resultados/efeitos a serem alcançados (em relação ao problema identificado e sob a perspectiva dos públicos envolvidos)**

O projeto visa como objetivo os três tópicos descritos a seguir:

1. Organização e Acessibilidade de Dados

O sistema visa organizar e centralizar as informações de clientes, serviços e atendimentos em um único banco de dados. Ao cadastrar os dados dos serviços, como tipo, preços e disponibilidade, o sistema torna a recuperação dessas informações rápida e acessível. Com SQL para armazenar e consultar os dados e Java para construir a interface e funcionalidades, o objetivo é simplificar a gestão do negócio, eliminando o uso de planilhas desorganizadas e documentos dispersos.

2. Agilidade e Automação de Processos

A automação dos processos de cadastramento de serviços e agendamento aumenta a agilidade do atendimento e reduz o esforço manual do esteticista. Com o sistema, é possível criar funcionalidades de atualização automática de agenda, controle de disponibilidade e envio de notificações ou lembretes aos clientes, agilizando a rotina de trabalho. Isso também facilita ajustes de última hora e evita duplicidade de cadastros ou erros na reserva de horários, melhorando a experiência do cliente.

3. Aprimoramento na Experiência do Usuário e Fidelização de Clientes

O sistema de cadastramento permite a personalização dos serviços e a consulta fácil ao histórico dos clientes, como tratamentos realizados e preferências pessoais. Com isso, a esteticista busca criar uma experiência mais personalizada e eficiente para cada cliente, reforçando a fidelização e satisfação. Além disso, a implementação de Java permite construir uma interface amigável e intuitiva, enquanto SQL proporciona consultas rápidas, contribuindo para um atendimento diferenciado e de qualidade.

- [Referencial teórico \(subsídio teórico para propositura de ações da extensão\)](#)

Referencial Teórico:

O uso de sistemas informatizados para o cadastramento e gerenciamento de dados de clientes na área de estética tem se tornado cada vez mais essencial, especialmente com o aumento da demanda por serviços personalizados e pela necessidade de otimização de processos. Sistemas de cadastro com Java e SQL são fundamentados em conceitos de bancos de dados relacionais, programação orientada a objetos e sistemas de informação, que proporcionam a eficiência e a automação necessárias para gerenciar com precisão informações sensíveis e rotinas do negócio.

Banco de Dados Relacional e SQL

Bancos de dados relacionais permitem a organização estruturada de dados em tabelas interconectadas, viabilizando uma gestão eficiente de informações e proporcionando

fácil recuperação de dados, atualização e manipulação conforme necessário (Elmasri & Navathe, 2016). SQL, ou Structured Query Language, é a linguagem padrão para manipulação de dados em bancos de dados relacionais, sendo amplamente utilizada para operações CRUD (Create, Read, Update, Delete) em sistemas de gerenciamento. Essas operações possibilitam que os dados de clientes, histórico de serviços e preferências sejam armazenados e acessados de maneira ágil e confiável, elementos cruciais para a gestão em negócios de estética.

Programação Orientada a Objetos com Java

Java é uma linguagem de programação orientada a objetos (OO) que oferece robustez, portabilidade e modularidade, características que tornam seu uso ideal em sistemas de gerenciamento e cadastro (Horstmann & Cornell, 2012). A Orientação a Objetos facilita a criação de estruturas de dados complexas, como classes e objetos, que representam clientes, serviços, produtos e agendamentos de forma organizada e intuitiva. Utilizando Java, é possível criar uma interface amigável e intuitiva para o usuário, permitindo uma gestão prática dos dados do cliente, além de oferecer recursos para integração com bancos de dados, como a API JDBC (Java Database Connectivity). A JDBC permite a conexão do Java a bancos SQL, possibilitando a execução de consultas e a manipulação de dados de clientes em tempo real (Deitel & Deitel, 2015).

Sistemas de Informação e Automação de Processos

Sistemas de cadastramento de clientes são classificados como Sistemas de Informação, pois auxiliam empresas na coleta, organização e análise de dados de forma estratégica, suportando a tomada de decisão (Laudon & Laudon, 2014). Em empresas de estética, o gerenciamento eficaz do relacionamento com o cliente por meio de um sistema informatizado proporciona uma operação mais ágil e personalizada, o que aumenta a satisfação e a fidelização. A automação dos processos por meio do sistema de cadastro permite o gerenciamento de agendas, controle de histórico de serviços e envio de lembretes, melhorando a eficiência e reduzindo a probabilidade de erros humanos.

Experiência do Usuário e Interface Gráfica

Um bom sistema de cadastramento deve prezar pela usabilidade e acessibilidade, visando sempre uma interface intuitiva e de fácil navegação (Nielsen & Budiu, 2013). Java oferece bibliotecas para construção de interfaces gráficas que facilitam a interação do usuário com o sistema. A adoção de uma boa experiência do usuário (UX) permite que esteticistas ou recepcionistas manipulem dados de clientes e serviços de maneira intuitiva, com fácil acesso ao histórico de atendimentos e preferências de cada cliente, contribuindo para uma experiência de atendimento diferenciada e de qualidade.

Referências Bibliográficas

- Deitel, P., & Deitel, H. (2015). Java: How to Program. Prentice Hall.
- Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems. Pearson.
- Horstmann, C. S., & Cornell, G. (2012). Core Java Volume I—Fundamentals. Prentice Hall.
- Laudon, K. C., & Laudon, J. P. (2014). Management Information Systems: Managing the Digital Firm. Pearson.
- Nielsen, J., & Budi, R. (2013). Mobile Usability. New Riders.

- **PLANEJAMENTO E DESENVOLVIMENTO DO PROJETO**

- **Plano de trabalho (usando ferramenta acordada com o docente)**

Semana 1: Busca pela parte interessada (responsáveis – Erick Freire e Sidney Rodrigues)

Atividades: Entrar em contato com as possíveis partes interessadas e com isso solicitando a assinatura da carta de apresentação caso a mesma decida participar do projeto.

Após a primeira parte afirmada, os integrantes do grupo responsáveis irão coletar as informações da problemática do estabelecimento, para decidirmos o andamento do projeto.

Semana 2: Iniciar o esqueleto do projeto (responsáveis – Daniel Pacheco e Isaac Blanco)

Atividades: Com os dados coletados na semana 1, os responsáveis iram começar a montar o código para estruturar o sistema e a sincronia com o banco de dados onde serão alocadas os dados cadastrados pelo sistema principal.

Semana 3: Correção de Bugs para entrega do projeto (responsáveis – Daniel Pacheco e Isaac Blanco)

Atividades: Semana 3 é focada totalmente para otimização do sistema e do banco de dados e com isso retirando ou minimizando os erros que a parte interessada possa vir a sofrer na utilização do sistema.

Semana 4: Conclusão do relatório (Responsável – Daniel Vinicius)

Atividades: Com a conclusão do sistema, a última semana será focada na finalização do relatório para a entrega final do trabalho, no relatório sendo descrito dados dos funcionamentos do projeto e informações sobre as semanas de produção.

- Descrição da forma de envolvimento do público participante na formulação do projeto, seu desenvolvimento e avaliação, bem como as estratégias pelo grupo para mobilizá-los.

O projeto acabou tendo duas reuniões, a primeira sendo para coleta de dados sobre a problemática da parte interessada e a reunião final para apresentação do resultado do projeto.

Primeira reunião - Dia: 14/10/2024

Na primeira reunião, conversamos com a parte interessada buscando saber quais eram as principais problemáticas e dificuldades que a mesma possuía no seu dia a dia, para em cima disso conseguirmos decidir como moldaríamos o projeto para se adequar do melhor jeito no dia de trabalho da responsável do estabelecimento.



Segunda reunião – Dia: 02/10/2024

Com a conclusão do projeto e finalização da apresentação para o professor da matéria, levamos todo o projeto criado para apresentar para a parte interessada, com isso explicamos todo o funcionamento do sistema para a mesma poder finalmente incluir o sistema para auxiliar e facilitar o seu dia a dia.

Grupo de trabalho (descrição da responsabilidade de cada membro)

Daniel Vinicius – RESPONSÁVEL PELO RELATÓRIO ESCRITO

Responsável por elaborar todo o relatório escrito ao final do projeto, com isso finalizando todas as entregas do projeto para finalização devida do projeto.

Erick Freire e Sidney Rodrigues – RESPONSÁVEIS PELO CONTATO COM A PARTE INTERESSADA

Responsáveis por identificar e estabelecer o contato entre o grupo com a parte interessada, com isso coordenando a passagem de informações sobre a parte interessada pro grupo e repassando os avanços para a mesma, com isso facilitando uma total compreensão entre as duas partes.

Daniel Pacheco e Isaac Blanco – RESPONSÁVEIS PELO DESENVOLVIMENTO DO CÓDIGO

Responsáveis pela criação, desenvolvimento e correção de bugs dos códigos para o sistema com aplicação em java e também para o banco de dados (SQL).

Com isso concluindo a construção do sistema para o projeto.

- [Metas, critérios ou indicadores de avaliação do projeto](#)

Metas do Projeto

1. Organizar e Centralizar Dados de Clientes e Serviços

- **Meta:** Desenvolver um banco de dados relacional utilizando SQL para armazenar e estruturar dados de clientes, serviços e agendamentos
- **Indicador:** Percentual de completude e precisão dos dados registrados.
- **Critério de Avaliação:** O banco de dados deve ser acessível e atualizado em tempo real, permitindo a consulta rápida de informações. Um teste inicial de preenchimento e consulta será realizado para garantir a consistência dos dados.

2. Automatizar Processos de Cadastramento e Agendamento

- **Meta:** Implementar funcionalidades que permitam o cadastro, consulta e atualização automatizada dos dados, incluindo agendamentos
- **Indicador:** Redução de tempo no processo de cadastramento e atualização de agendamentos.
- **Critério de Avaliação:** O sistema deve permitir o cadastramento de clientes e agendamentos em menos de 2 minutos por operação. Um teste de usabilidade será conduzido com os usuários para medir a eficiência e a satisfação no uso da funcionalidade.

3. Aprimorar a Experiência do Usuário

- **Meta:** Desenvolver uma interface gráfica em Java que seja intuitiva e fácil de usar, proporcionando um fluxo de navegação simplificado.

- **Indicador:** Nível de satisfação do usuário em relação à usabilidade e acessibilidade da interface.

- **Critério de Avaliação:** Realizar um teste de usabilidade com usuários representativos (esteticista) e coletar feedback sobre a clareza e navegabilidade da interface. Espera-se obter uma pontuação de pelo menos 85% de aprovação.

- Recursos previstos

Suporte institucional: Acesso aos laboratórios de informática da instituição para programar e testar o sistema, além de suporte para divulgação do projeto na comunidade e auxílio logístico, como salas para oficinas e reuniões.

Recursos humanos: Os membros do grupo de trabalho responsáveis pela instalação, configuração e teste dos sensores, além do suporte técnico de professores e profissionais especializados, caso necessário.

- Detalhamento técnico do projeto

Arquitetura e Tecnologias

O sistema é baseado em uma arquitetura cliente-servidor com integração via JDBC para conectar a aplicação Java ao banco de dados SQL. A interface gráfica é desenvolvida em Java Swing, enquanto o banco de dados utiliza SQL para operações de cadastro, consulta e atualização.

Funcionalidades

Cadastro de Clientes: Interface para registrar informações como nome, contato e histórico de atendimento.

Cadastro de Serviços: Permite registrar serviços oferecidos com detalhes como preço e duração.

Agendamento de Serviços: Seleção de clientes e horários disponíveis para agendamento.

Consulta ao Histórico: Exibe o histórico de atendimentos para um atendimento mais personalizado.

Avaliação

Critérios de avaliação incluem:

Precisão dos Dados: Taxa de sucesso nos cadastros.

Tempo de Resposta: Operações em menos de 2 segundos.

Satisfação do Usuário: Pontuação mínima de 85% em testes de usabilidade.

Testes Realizados

Foram feitos testes funcionais, de usabilidade e de desempenho para garantir a consistência e a eficiência do sistema.

Esse sistema foi projetado para otimizar o atendimento em clínicas de estética, garantindo rapidez, segurança e facilidade no uso.

- ENCERRAMENTO DO PROJETO

- Relato Coletivo:

Como grupo, consideramos que o projeto atingiu plenamente os objetivos sociocomunitários, ao criar um sistema de cadastramento de clientes voltado para a organização e a eficiência em clínicas de estética. A solução desenvolvida com Java e SQL permite à esteticista centralizar dados, simplificar agendamentos e personalizar o atendimento, alinhando-se às necessidades de pequenos empreendimentos que desejam profissionalizar sua gestão de clientes. Durante o processo, o grupo colaborou em todas as etapas, desde a modelagem dos dados até a construção da interface, com um enfoque em tornar o sistema intuitivo e prático. As principais aprendizagens envolvem tanto o desenvolvimento técnico quanto a compreensão de como a tecnologia pode beneficiar pequenos negócios na área de estética.

- Avaliação de reação da parte interessada

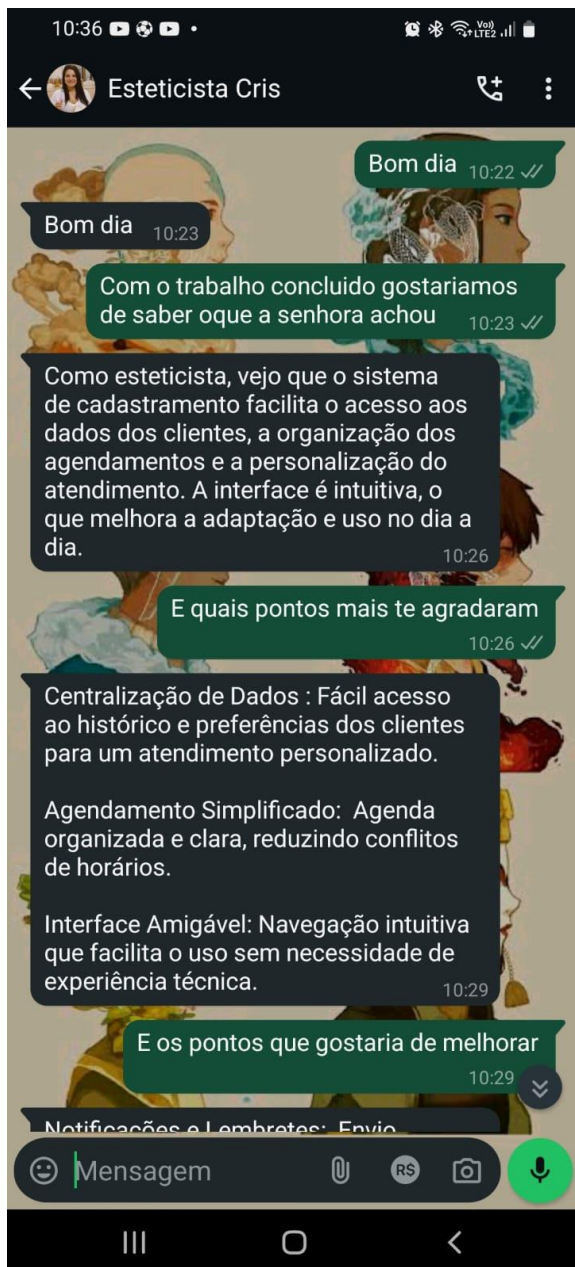
Como esteticista, vejo que o sistema de cadastramento facilita o acesso aos dados dos clientes, a organização dos agendamentos e a personalização do atendimento. A interface é intuitiva, o que melhora a adaptação e uso no dia a dia.

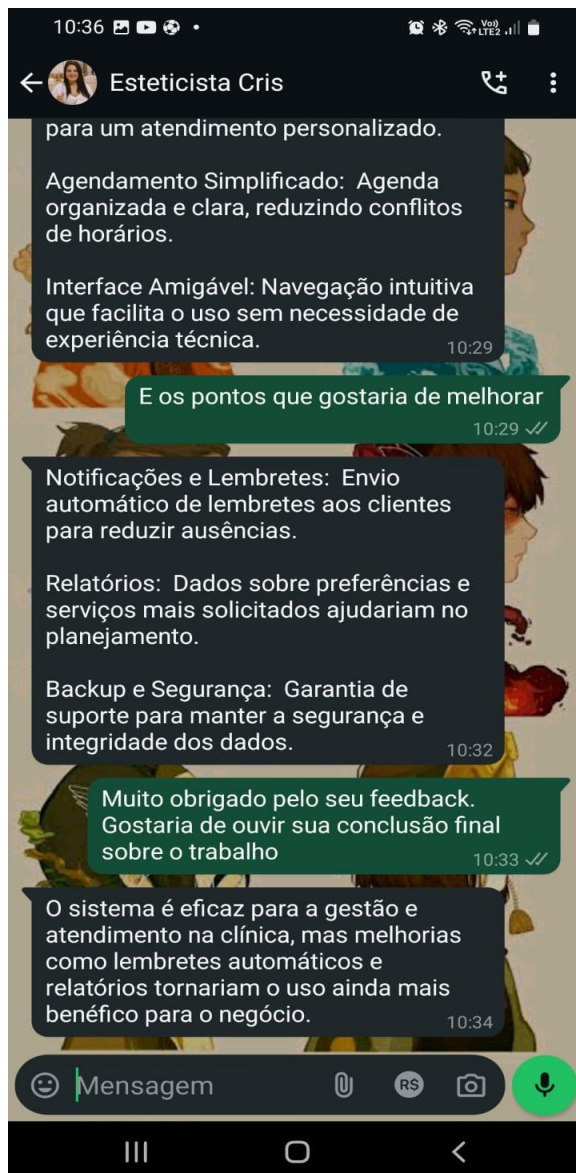
Pontos Positivos

- Agendamento Simplificado: Agenda organizada e clara, reduzindo conflitos de horários.
- Interface Amigável: Navegação intuitiva que facilita o uso sem necessidade de experiência técnica.

Pontos a Melhorar

- Notificações e Lembretes: Envio automático de lembretes aos clientes para reduzir ausências.
- Relatórios: Dados sobre preferências e serviços mais solicitados ajudariam no planejamento.
- Backup e Segurança: Garantia de suporte para manter a segurança e integridade dos dados.





- Relato de Experiência Individual

- CONTEXTUALIZAÇÃO

-Daniel Vinicius Lima Caravelos: Minha participação foi principalmente na construção de todo o relatório escrito, compilando as informações captadas com o passar das semanas para construir um documento conciso e direto de acordo com os dados solicitados.

Minha experiência foi positiva, ver toda a construção do projeto em poucas semanas acabou sendo um tanto desafiadora, porém muito útil para a minha construção profissional.

-Sidney Rodrigues da Costa Junior: Minha participação neste projeto foi na busca na parte interessada, junto com o erick, ficamos alguns dias procurando alguém que fosse aceitar nossa proposta, muita gente recusou pelo medo de aceitar algo que eles não teriam que pagar nada.

Minha experiencia foi positiva, a gente ficou procurando bastante mas no final a gente achou alguém que fosse cooperativo enquanto teve a criação do produto.

-Daniel Lima Pacheco Do Nascimento: Minha participação foi no desenvolvimento do projeto, onde tive a oportunidade de contribuir ativamente e de auxiliar em diversas etapas. A experiência foi extremamente enriquecedora: cada erro foi uma oportunidade de aprendizado, assim como cada acerto. Ao longo do processo, aprendi a lidar com situações desafiadoras e a tirar lições valiosas de todas elas. Sem dúvida, essa vivência foi fundamental para o meu crescimento profissional.

-Erick Freire dos Santos: Minha contribuição no projeto foi fundamental na fase inicial de levantamento de informações e definição do escopo. Especificamente, fui responsável por identificar e abordar uma parte interessada chave para o sucesso do projeto. Minha principal tarefa foi entrar em contato com a parte interessada, realizar uma conversa estruturada com essa pessoa e entender suas expectativas, necessidades e perspectivas em relação ao projeto.

-Isaac Blanco: Atuei na parte lógica do projeto (código). Com a avaliação constante da parte interessada e apoio do grupo, foi possível lidar com os erros que ocorreram e realizar otimizações. Acredito que não foi apenas um projeto, mas adicionou à minha pessoa e ao meu lado profissional.

- **METODOLOGIA**

-Daniel Vinicius Lima Caravelos: A experiência foi vivenciada em um ambiente acadêmico, onde desenvolvemos um sistema de cadastramento de clientes para uma clínica de estética, com foco em automação de dados e organização de agendamentos. O público envolvido incluiu esteticistas e o instrutor que forneceram feedback sobre as necessidades reais de um sistema de gestão de clientes. Durante o semestre, seguimos etapas de análise de requisitos, modelagem de dados, programação em Java e SQL e testes. Como integrante do grupo, contribuí principalmente na construção do relatório final do trabalho.

-Sidney Rodrigues da Costa Junior: A experiência foi vivenciada em um ambiente acadêmico, onde decidimos como que iremos prosseguir com o projeto, além de mudar certas partes do projeto com as necessidades da parte interessada, foi bem difícil conseguir decidir em algo que ira ajudar tanto a gente como a parte interessada, mas no final conseguimos.

-Daniel Lima Pacheco Do Nascimento: A experiência ocorreu em um ambiente acadêmico, no qual colaborei no desenvolvimento de um sistema voltado para a área de estética. O objetivo principal foi criar uma solução que facilitasse a gestão de

clientes, o controle de agendamentos, os procedimentos realizados e os valores de maneira simples, rápida e intuitiva, visando otimizar o trabalho na área.

-Erick Freire dos Santos: A experiência foi vivenciada em dois ambientes distintos. O primeiro, um ambiente acadêmico, foi onde desenvolvi o projeto, com o apoio de colegas e orientadores, enquanto o segundo foi uma sala de estética, que funcionou como o espaço da parte interessada no projeto. Esta parte interessada era uma profissional da área de estética que desempenhou um papel crucial no feedback do desenvolvimento do projeto. O ambiente acadêmico foi o ponto de partida para a elaboração das ideias, enquanto a sala de estética foi o local onde os encontros com a parte interessada ocorreram, permitindo um espaço prático para discussões e ajustes do projeto.

-Isaac Blanco: Fiquei responsável pela parte do código do programa. Foi uma experiência desafiadora e empolgante lidar com erros de lógica e fazer as alterações necessárias para o melhor funcionamento do mesmo. O resultado esperado foi alcançado de forma relativamente rápida, com uma intensidade maior nos últimos dias antes da entrega do projeto. A satisfação da parte interessada foi alcançada graças aos membros do grupo, em conjunto, e a orientação do instrutor.

- **RESULTADOS E DISCUSSÃO:**

-Daniel Vinicius Lima Caravelos: Como responsável pela construção do relatório escrito, minha experiência foi enriquecedora ao transformar o desenvolvimento técnico do sistema em um documento claro e detalhado. A organização das etapas e a descrição dos objetivos foram facilitadas pela colaboração com a equipe e o acesso a informações detalhadas sobre o sistema. Aprendi a importância de traduzir termos técnicos para uma linguagem acessível e coesa. As principais dificuldades envolveram entender plenamente os aspectos técnicos para descrevê-los de forma precisa, mas isso contribuiu para um aprendizado sólido sobre o projeto e suas funcionalidades.

-Sidney Rodrigues da Costa Junior: Foi muito estressante ter que ficar obtendo informação e ajudando o grupo para o projeto sair da maneira certa, mas no final o produto saiu da maneira que a parte interessada queria e tudo deu certo no final.

-Daniel Lima Pacheco Do Nascimento: O desenvolvimento desse projeto foi extremamente enriquecedor para o meu amadurecimento profissional. Cada erro ao longo do caminho foi uma oportunidade para crescer e aprender. Embora tenha enfrentado desafios, o apoio dos meus colegas foi crucial para superar as dificuldades e transformar esses momentos em aprendizado.

-Erick Freire dos Santos: Antes de iniciar o projeto, minha principal expectativa era que encontrar uma parte interessada fosse um desafio mais complicado. Eu imaginava que seria difícil identificar alguém disposto a colaborar ou que estivesse realmente interessado no projeto. No entanto, essa expectativa foi superada de forma positiva. Ao procurar uma profissional da área de estética, logo encontrei uma parte interessada que estava não só disposta a participar, mas também engajada no processo, fornecendo um feedback extremamente valioso.

-Isaac Blanco: Pela função que me foi dada, houve dificuldades em fazer com que o programa fosse amigável, impossibilitasse alguns erros que pudessem ocorrer – tanto erros humanos quanto erros do programa em si – e a difícil adaptação à linguagem utilizadas. Essas dificuldades foram difíceis na mesma medida que foram prazerosas de superar. Sem dúvidas o projeto mudou a forma que enxergo trabalhar em grupo e programação.

- REFLEXÃO APROFUNDADA

-Daniel Vinicius Lima Caravelos: Ao focar na construção do relatório final do projeto, percebi como a teoria precisa ser adaptada para uma apresentação clara e acessível. Embora os conceitos de organização e automação de dados sejam amplamente discutidos, traduzir a complexidade técnica do sistema para uma linguagem compreensível foi desafiador. Essa experiência ressaltou a importância de conectar teoria e prática de forma coesa, visando atender tanto às expectativas dos leitores quanto às necessidades do usuário final. Trabalhar no relatório me proporcionou um entendimento mais profundo dos impactos práticos do sistema para o público-alvo e a importância da clareza na comunicação escrita.

-Sidney Rodrigues da Costa Junior: Achei que iria ser mais fácil achar uma parte interessada, mas até que foi bem difícil, deveria ter me preparado melhor para um problema desses, mas conseguimos achar alguém dentro da primeira semana, a outra parte difícil foi ficar passando as informações da parte interessada para a nossa equipe, mas felizmente deu tudo certo no final.

-Daniel Lima Pacheco Do Nascimento: Durante todo o processo de desenvolvimento, percebi que a prática constante é fundamental para meu crescimento. Além disso, o estudo contínuo sobre metodologias de projetos e arquiteturas de sistemas tem se mostrado essencial para a criação de soluções mais eficientes e bem estruturadas.

-Erick Freire dos Santos: Ao longo do desenvolvimento deste projeto, houve uma forte interação entre o que foi aprendido teoricamente e o que foi vivenciado na prática. A teoria forneceu as bases e os conceitos fundamentais para o desenvolvimento do projeto, mas a aplicação prática revelou nuances e desafios que não estavam completamente antecipados pela teoria.

-Isaac Blanco: A ideia era criar um sistema a fim de demonstrar o que a tecnologia pode fazer pelo negócio das pessoas. Considero que o propósito foi cumprido e a expectativa não se difere da realidade. As ideias foram executadas, a parte interessada se sentiu satisfeita e houve aprendizado para todos.

- **CONSIDERAÇÕES FINAIS**

-Daniel Vinicius Lima Caravelos: Considero que o projeto trouxe resultados satisfatórios para organizar o cadastramento e agendamento de clientes em clínicas de estética. Em projetos futuros, seria interessante explorar integrações com aplicativos de notificação para lembretes automáticos aos clientes, além de relatórios detalhados que ofereçam insights sobre as preferências e frequência de serviços. Outras soluções tecnológicas, como um aplicativo mobile, poderiam expandir o acesso e a praticidade do sistema, proporcionando ainda mais flexibilidade ao usuário. Esse projeto também abre possibilidades de pesquisas sobre o impacto da digitalização em pequenos negócios de estética, fortalecendo o uso da tecnologia em novos segmentos.

-Sidney Rodrigues da Costa Junior: Eu me senti satisfeito após nos termos terminado o projeto, fiz novas amizades, aprendi coisas novas, e me diverti bastante durante toda essa experiencia, eu poderia ter sido mais proativo em questão da parte interessada, mas deu tudo certo no final.

-Daniel Lima Pacheco Do Nascimento: Em projetos futuros, uma possibilidade seria a criação de um aplicativo móvel para complementar o sistema. Isso permitiria aos usuários realizar agendamentos e pagamentos de forma remota, trazendo mais flexibilidade e comodidade tanto para os usuários quanto para seus clientes.

-Erick Freire dos Santos: Ao refletir sobre os outros aspectos que podem ser trabalhados junto à parte interessada, bem como as perspectivas de trabalhos futuros, é possível explorar tanto áreas de extensão quanto de pesquisa, considerando o que foi aprendido durante o desenvolvimento do projeto e as necessidades práticas que surgiram. Também é interessante pensar em soluções tecnológicas alternativas que poderiam ter sido

implementadas para melhorar ou expandir o impacto do projeto. Um aspecto importante seria monitorar o impacto das mudanças implementadas ao longo do tempo. Isso poderia envolver a coleta de feedback contínuo da parte interessada, para avaliar o desempenho das soluções aplicadas e identificar melhorias adicionais.

-Isaac Blanco: Futuramente, juntos à parte interessada, pode haver melhorias, como registros com mais informações e a possibilidade de serem tomadas mais ações com os pacientes já registrados no sistema. Expandir o programa, num geral, de acordo com a eficiência do programa e interação constante e direta com a parte interessada.

Segue imagens do andamento do trabalho durante as semanas e da apresentação final para o professor:





Clínica - Gerenciamento de Pacientes

Mensagem

 **Paciente encontrado:**
Nome: Isaac, CPF: 12345678900
Data da Consulta: 25/12/24
Procedimentos:
- Peeling químico (R\$ 200.0)

OK

Nome:

CPF:

Data da Consulta (dd/MM/yy):

Procedimento:

Buscar **Adicionar** **Remover** **Editar**

Clínica - Gerenciamento de Pacientes

Mensagem

 **Paciente adicionado com sucesso!**

OK

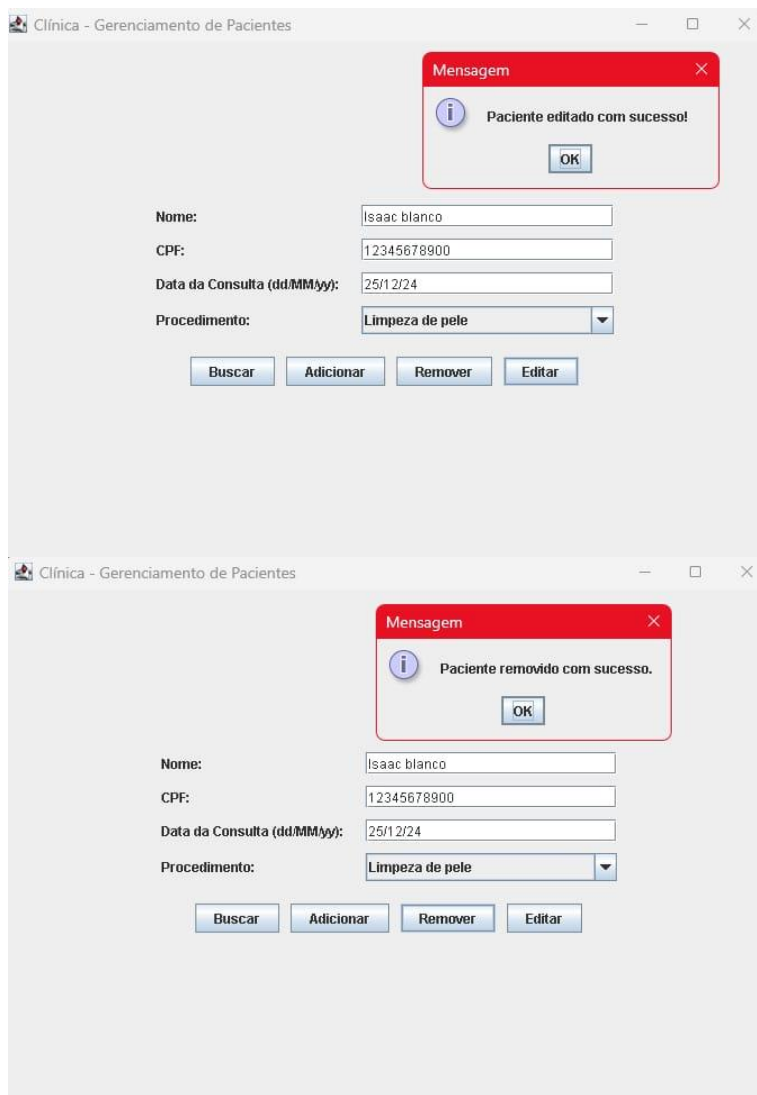
Nome:

CPF:

Data da Consulta (dd/MM/yy):

Procedimento:

Buscar **Adicionar** **Remover** **Editar**



Segue códigos utilizados no trabalho:

BANCO DE DADOS:

```
package javatrab1;
```

```
import java.sql.*;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class BancoDeDados {
```

```
    private static final String URL = "jdbc:sqlite:sistemaRegistro.db";
```

```
    public BancoDeDados() {
```

```
        criarTabelaPaciente();
```

```
        criarTabelaProcedimento();
```

```
}
```

```
private void criarTabelaPaciente() {  
    String sql = "CREATE TABLE IF NOT EXISTS Paciente (" +  
        "nome TEXT NOT NULL, " +  
        "cpf TEXT PRIMARY KEY, " +  
        "dataConsulta TEXT NOT NULL" +  
        ");";  
    try (Connection conn = DriverManager.getConnection(URL);  
        Statement stmt = conn.createStatement()) {  
        stmt.execute(sql);  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

```
private void criarTabelaProcedimento() {  
    String sql = "CREATE TABLE IF NOT EXISTS Procedimento (" +  
        "id INTEGER PRIMARY KEY AUTOINCREMENT," +  
        "cpfPaciente TEXT NOT NULL," +  
        "nome TEXT NOT NULL," +  
        "valor REAL NOT NULL," +  
        "FOREIGN KEY(cpfPaciente) REFERENCES Paciente(cpf)" +  
        ");";  
    try (Connection conn = DriverManager.getConnection(URL);  
        Statement stmt = conn.createStatement()) {  
        stmt.execute(sql);  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

```
public void registrarPacienteNoBanco(Paciente paciente) {  
    String sql = "INSERT INTO Paciente (nome, cpf, dataConsulta) VALUES  
(?, ?, ?)";  
}
```

```

try (Connection conn = DriverManager.getConnection(URL);
    PreparedStatement pstmt = conn.prepareStatement(sql)) {

    pstmt.setString(1, paciente.getNome());
    pstmt.setString(2, paciente.getCpf());
    pstmt.setString(3, paciente.getDataConsulta().toString());

    pstmt.executeUpdate();
} catch (SQLException e) {
    e.printStackTrace();
}

for (Procedimento p : paciente.getProcedimentos()) {
    adicionarProcedimentoAoPaciente(paciente.getCpf(), p);
}

}

public List<Paciente> listarPacientes() {
    List<Paciente> pacientes = new ArrayList<>();
    String sql = "SELECT nome, cpf, dataConsulta FROM Paciente";

    try (Connection conn = DriverManager.getConnection(URL);
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        while (rs.next()) {
            String nome = rs.getString("nome");
            String cpf = rs.getString("cpf");
            String dataConsulta = rs.getString("dataConsulta");

            Paciente paciente = new Paciente(nome, cpf,
Date.valueOf(dataConsulta));

paciente.getProcedimentos().addAll(buscarProcedimentosDoPaciente(cpf)
);

```

```

        pacientes.add(paciente);
    }
} catch (SQLException e) {
    e.printStackTrace();
}

return pacientes;
}

public Paciente buscarPaciente(String cpf) {
    String sql = "SELECT nome, cpf, dataConsulta FROM Paciente WHERE
cpf = ?";
    Paciente paciente = null;

    try (Connection conn = DriverManager.getConnection(URL);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, cpf);
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            String nome = rs.getString("nome");
            String dataConsulta = rs.getString("dataConsulta");

            paciente = new Paciente(nome, cpf, Date.valueOf(dataConsulta));

            paciente.getProcedimentos().addAll(buscarProcedimentosDoPaciente(cpf)
);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }

    return paciente;
}

```

```

public void removerPaciente(String cpf) {
    String sqlPaciente = "DELETE FROM Paciente WHERE cpf = ?";
    String sqlProcedimentos = "DELETE FROM Procedimento WHERE
cpfPaciente = ?";

    try (Connection conn = DriverManager.getConnection(URL);
        PreparedStatement pstmtPaciente =
conn.prepareStatement(sqlPaciente);
        PreparedStatement pstmtProcedimentos =
conn.prepareStatement(sqlProcedimentos)) {

        pstmtProcedimentos.setString(1, cpf);
        pstmtProcedimentos.executeUpdate();

        pstmtPaciente.setString(1, cpf);
        pstmtPaciente.executeUpdate();

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public boolean pacienteExiste(String cpf) {
    String sql = "SELECT COUNT(*) FROM Paciente WHERE cpf = ?";
    try (Connection conn = DriverManager.getConnection(URL);
        PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, cpf);
        ResultSet rs = pstmt.executeQuery();
        return rs.getInt(1) > 0;
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

```

```

    public void adicionarProcedimentoAoPaciente(String cpf, Procedimento
procedimento) {
        String sql = "INSERT INTO Procedimento (cpfPaciente, nome, valor)
VALUES (?, ?, ?)";
        try (Connection conn = DriverManager.getConnection(URL);
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, cpf);
            pstmt.setString(2, procedimento.getNome());
            pstmt.setDouble(3, procedimento.getValor());

            pstmt.executeUpdate();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

```

    public List<Procedimento> buscarProcedimentosDoPaciente(String cpf) {
        List<Procedimento> procedimentos = new ArrayList<>();
        String sql = "SELECT nome, valor FROM Procedimento WHERE
cpfPaciente = ?";

        try (Connection conn = DriverManager.getConnection(URL);
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, cpf);
            ResultSet rs = pstmt.executeQuery();

            while (rs.next()) {
                String nome = rs.getString("nome");
                double valor = rs.getDouble("valor");
                procedimentos.add(new Procedimento(nome, valor));
            }
        } catch (SQLException e) {

```

```

        e.printStackTrace();
    }

    return procedimentos;
}
}

```

GUI Loja:

```

package javatrab1;

import javax.swing.*.*;
import java.awt.*.*;
import java.sql.Date;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.List;

public class GUILoja extends JFrame {
    private JTextField nomeField, cpfField, dataConsultaField;
    private JButton buscarButton, adicionarButton, removerButton,
editarButton;
    private JComboBox<String> procedimentoComboBox;
    private BancoDeDados banco;
    private List<Procedimento> procedimentos;

    public GUILoja() {
        super("Clínica - Gerenciamento de Pacientes");
        banco = new BancoDeDados();
        procedimentos = criarListaDeProcedimentos(); // Cria a lista de
procedimentos
        initUI();
    }

    private void initUI() {

```

```
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setSize(700, 500);
setLocationRelativeTo(null);

JPanel mainPanel = new JPanel(new GridBagLayout());
GridBagConstraints gbc = new GridBagConstraints();
gbc.insets = new Insets(5, 10, 5, 10);
gbc.fill = GridBagConstraints.HORIZONTAL;

// Campo de Nome
JLabel nomeLabel = new JLabel("Nome:");
gbc.gridx = 0;
gbc.gridy = 0;
mainPanel.add(nomeLabel, gbc);

nomeField = new JTextField(20);
gbc.gridx = 1;
gbc.gridy = 0;
mainPanel.add(nomeField, gbc);

// Campo de CPF
JLabel cpfLabel = new JLabel("CPF:");
gbc.gridx = 0;
gbc.gridy = 1;
mainPanel.add(cpfLabel, gbc);

cpfField = new JTextField(15);
gbc.gridx = 1;
gbc.gridy = 1;
mainPanel.add(cpfField, gbc);

// Campo de Data de Consulta
JLabel dataConsultaLabel = new JLabel("Data da Consulta
(dd/MM/yy):");
gbc.gridx = 0;
```

```
gbc.gridy = 2;
mainPanel.add(dataConsultaLabel, gbc);

dataConsultaField = new JTextField(10);
gbc.gridx = 1;
gbc.gridy = 2;
mainPanel.add(dataConsultaField, gbc);

// ComboBox de Procedimento
JLabel procedimentoLabel = new JLabel("Procedimento:");
gbc.gridx = 0;
gbc.gridy = 3;
mainPanel.add(procedimentoLabel, gbc);

procedimentoComboBox = new JComboBox<>();
preencherProcedimentosComboBox(); // Preenche inicialmente com
todos os procedimentos
gbc.gridx = 1;
gbc.gridy = 3;
mainPanel.add(procedimentoComboBox, gbc);

// Painel de botões de ação
JPanel panelAcoes = new JPanel(new FlowLayout(FlowLayout.CENTER,
10, 10));

buscarButton = new JButton("Buscar");
buscarButton.addActionListener(e -> buscarPaciente());
panelAcoes.add(buscarButton);

adicionarButton = new JButton("Adicionar");
adicionarButton.addActionListener(e -> adicionarPaciente());
panelAcoes.add(adicionarButton);

removerButton = new JButton("Remover");
removerButton.addActionListener(e -> removerPaciente());
```

```

panelAcoes.add(removerButton);

editarButton = new JButton("Editar");
editarButton.addActionListener(e -> editarPaciente());
panelAcoes.add(editarButton);

gbc.gridx = 0;
gbc.gridy = 4;
gbc.gridwidth = 2;
mainPanel.add(panelAcoes, gbc);

add(mainPanel, BorderLayout.CENTER);
setVisible(true);
}

private List<Procedimento> criarListaDeProcedimentos() {
    List<Procedimento> lista = new ArrayList<>();
    lista.add(new Procedimento("Tratamento de estrias", 100.0));
    lista.add(new Procedimento("Drenagem linfática", 80.0));
    lista.add(new Procedimento("Preenchimento labial", 150.0));
    lista.add(new Procedimento("Peeling químico", 200.0));
    lista.add(new Procedimento("Limpeza de pele", 60.0));
    lista.add(new Procedimento("Tratamento de varizes", 80.0));
    return lista;
}

private void preencherProcedimentosComboBox() {
    procedimentoComboBox.removeAllItems();
    for (Procedimento procedimento : procedimentos) {
        procedimentoComboBox.addItem(procedimento.getNome());
    }
}

private void atualizarProcedimentosComboBox(List<Procedimento>
procedimentosPaciente) {

```

```

procedimentoComboBox.removeAllItems();

// Adiciona o procedimento do paciente como o primeiro item da
ComboBox
for (Procedimento proc : procedimentosPaciente) {
    procedimentoComboBox.addItem(proc.getNome() + " (R$ " +
proc.getValor() + ")");
}

// Adiciona todos os procedimentos disponíveis em seguida
for (Procedimento procedimento : procedimentos) {
    if (procedimentosPaciente.stream().noneMatch(p ->
p.getNome().equals(procedimento.getNome())) {
        procedimentoComboBox.addItem(procedimento.getNome());
    }
}
}

private String formatarData(Date data) {
    SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yy");
    return sdf.format(data);
}

private Date converterStringParaData(String dataStr) throws
ParseException {
    SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yy");
    java.util.Date utilDate = sdf.parse(dataStr);
    return new Date(utilDate.getTime());
}

private void buscarPaciente() {
    String cpf = cpfField.getText().trim();
    if (cpf.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Digite um CPF para
buscar.");
    }
}

```

```

        return;
    }

    Paciente paciente = banco.buscarPaciente(cpf);
    if (paciente == null) {
        JOptionPane.showMessageDialog(this, "Paciente não encontrado.");
    } else {
        nomeField.setText(paciente.getNome());

dataConsultaField.setText(formatarData(paciente.getDataConsulta())); //
Formata a data

        // Combobox atualizada
        atualizarProcedimentosComboBox(paciente.getProcedimentos());

        JOptionPane.showMessageDialog(this, "Paciente encontrado:\n" +
paciente.toString());
    }
}

private void adicionarPaciente() {
    String nome = nomeField.getText().trim();
    String cpf = cpfField.getText().trim();
    String dataConsultaStr = dataConsultaField.getText().trim();

    if (nome.isEmpty() || cpf.isEmpty() || dataConsultaStr.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Todos os campos devem ser
preenchidos.");
        return;
    }
    if (banco.pacienteExiste(cpf)){
        JOptionPane.showMessageDialog(this,"Já existe este paciente nos
registros");
        return;
    }
}

```

```

try {
    Date dataConsulta = converterStringParaData(dataConsultaStr); //
    Converte a data do formato dd/MM/aa
    Paciente paciente = new Paciente(nome, cpf, dataConsulta);

    String procedimentoNome = (String)
procedimentoComboBox.getSelectedItemAt();
    Procedimento procedimentoSelecioneado = procedimentos.stream()
        .filter(p -> p.getNome().equals(procedimentoNome))
        .findFirst()
        .orElse(null);

    if (procedimentoSelecioneado != null) {
        paciente.adicionarProcedimento(procedimentoSelecioneado);
    }

    banco.registrarPacienteNoBanco(paciente);
    JOptionPane.showMessageDialog(this, "Paciente adicionado com
sucesso!");
    } catch (ParseException e) {
        JOptionPane.showMessageDialog(this, "Data de consulta inválida.
Use o formato dd/MM/yy.");
    }
}

private void removerPaciente() {
    String cpf = cpfField.getText().trim();
    if (cpf.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Digite um CPF para
remover.");
        return;
    }
}

```

```

        banco.removerPaciente(cpf);
        JOptionPane.showMessageDialog(this, "Paciente removido com
sucesso.");
    }

    private void editarPaciente() {
        String nome = nomeField.getText().trim();
        String cpf = cpfField.getText().trim();
        String dataConsultaStr = dataConsultaField.getText().trim();

        if (nome.isEmpty() || cpf.isEmpty() || dataConsultaStr.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Todos os campos devem ser
preenchidos.");
            return;
        }

        if (!banco.pacienteExiste(cpf)) {
            JOptionPane.showMessageDialog(this, "Paciente não encontrado
para edição.");
            return;
        }

        try {
            Date dataConsulta = converterStringParaData(dataConsultaStr); //
Converte a data do formato dd/MM/aa
            Paciente paciente = new Paciente(nome, cpf, dataConsulta);

            banco.removerPaciente(cpf);
            banco.registrarPacienteNoBanco(paciente);
            JOptionPane.showMessageDialog(this, "Paciente editado com
sucesso!");
        } catch (ParseException e) {
            JOptionPane.showMessageDialog(this, "Data de consulta inválida.
Use o formato dd/mm/aa.");
        }
    }

```

```

    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new GUILoja());
    }
}

```

PACIENTE:

```

package javatrab1;

import java.sql.Date;
import java.util.ArrayList;
import java.text.SimpleDateFormat;

public class Paciente extends Pessoa {
    private Date dataConsulta;
    private ArrayList<Procedimento> procedimentos; // Lista de
    procedimentos

    public Paciente(String nome, String cpf, Date dataConsulta) {
        super(nome, cpf); // Chama o construtor da superclasse
        this.dataConsulta = dataConsulta;
        this.procedimentos = new ArrayList<>(); // Inicializa a lista de
    procedimentos
    }

    public Date getDataConsulta() {
        return dataConsulta;
    }

    public void setDataConsulta(Date dataConsulta) {
        this.dataConsulta = dataConsulta;
    }

    public ArrayList<Procedimento> getProcedimentos() {

```

```

        return procedimentos;
    }

    public void adicionarProcedimento(Procedimento procedimento) {
        this.procedimentos.add(procedimento);
    }

    private String formatarData(Date data) {
        SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yy");
        return sdf.format(data);
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append(super.toString()).append("\n") // Chama o toString da
        classe Pessoa
        .append("Data da Consulta:
        ").append(formatarData(dataConsulta)).append("\n") // Formata a data
        .append("Procedimentos: ");

        for (Procedimento p : procedimentos) {
            sb.append("\n - ").append(p.toString());
        }

        return sb.toString();
    }
}

```

Pessoa:

```

package javatrab1;

public abstract class Pessoa {
    private String nome;
    private String cpf;

```

```

public Pessoa(String nome, String cpf) {
    this.nome = nome;
    this.cpf = cpf;
}

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public String getCpf() {
    return cpf;
}

public void setCpf(String cpf) {
    this.cpf = cpf;
}

@Override
public String toString() {
    return "Nome: " + nome + ", CPF: " + cpf;
}
}

```

PROCEDIMENTO:

```

package javatrab1;

public class Procedimento {
    private String nome;
    private double valor;
}

```

```
public Procedimento(String nome, double valor) {
    this.nome = nome;
    this.valor = valor;
}

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public double getValor() {
    return valor;
}

public void setValor(double valor) {
    this.valor = valor;
}

@Override
public String toString() {
    return nome + " (R$ " + valor + ")";
}
}
```